CS781 Capstone Project: In-House Legal Intelligence System Pretrained Legal Language Models and Parameter-Efficient Domain Adaptation

Ujjwal Prakash Roll Number: **241111**
ujjwal24@iitk.ac.in
Department of Computer Science and Engineering
Indian Institute of Technology Kanpur (IIT Kanpur)

November 2026

Abstract

Indian legal practice restricts sending privileged case records and unredacted filings to commercial cloud APIs due to Bar Council confidentiality rules and statutory privacy constraints under the DPDPA (2023). In this capstone, I built a self-hosted legal NLP pipeline across seven distinct phases. Operating within a strict 50M–300M parameter budget without external pretrained model weights, I trained four foundation architectures from scratch on Indian legal text: standard GPT-2, RoPE LLaMA with RMSNorm and SwiGLU, partial-RoPE GPT-NeoX, and a 24-layer Deep-Thin transformer. RoPE LLaMA converged best, reaching an evaluation loss of 2.731 (15.35 perplexity). I transferred this foundation to six downstream tasks using parameter-efficient fine-tuning (LoRA, LORMA, BitFit, Layer-wise) and knowledge distillation: Named Entity Recognition, Rhetorical Role Labeling, Judgment Prediction, Statute Tagging, Summarization, and Prior Case Retrieval. Threshold calibration on validation logits, multi-gram lexical indexing, and consensus ensembling produced consistent gains across the official evaluation benchmarks.

1 Competition Result

Codabench username: `ujjwalprakashgoodgod` / `a_241111` **Roll number:** 241111

GitHub Repository: https://github.com/ujjwalPrakash-spike/cs781-legal-intelligence

Table 1 lists the evaluation scores and leaderboard standing recorded across all seven phases. I ran 42 valid submissions during development, exceeding the required minimum of four submissions per phase.

Total valid Codabench submissions across all phases: 42 **Date ranks recorded:** November 7, 2026. Every phase met the four-submission requirement. Submitted variants covered Full Fine-Tuning, LoRA, LORMA, Layer-Wise freezing, BitFit, Knowledge Distillation, and multi-model consensus ensembles.

2 Introduction and Problem Description

Indian judicial proceedings generate large volumes of unstructured case law across the Supreme Court, High Courts, and appellate tribunals. While proprietary cloud LLMs generate fluent text, deploying them in legal practice creates genuine compliance barriers. The Advocates Act (1961), Bar Council rules on professional secrecy, and the Digital Personal Data Protection Act (DPDPA 2023) prohibit transmitting non-public case records, draft petitions, and privileged client communications to external multi-tenant infrastructure. For legal firms and judicial registries, language models must run locally on modest in-house hardware.

Table 1: Official phase evaluation scores, primary metrics, and valid submission counts recorded across all seven development phases.

Phase	Task Description	Primary Metric	Dev Score	Official Metric	Valid Subs
1	Legal LM Pretraining	Perplexity ↓ / BLEU-4 ↑	15.35 (PPL)	2.731 (Loss)	4
2	Legal Named Entity Recognition	Strict Micro-F1 ↑	0.6842	0.6710	6
3	Rhetorical Role Prediction	Macro-F1 ↑	0.8124	0.7985	7
4	Court Judgment Prediction	Macro-F1 ↑	0.6932	0.6815	7
5	Legal Statute Identification	Macro-F1 ↑	0.3338	0.3294	6
6	Abstractive Summarization	BERTScore / ROUGE-L ↑	0.3840 (R-L)	0.8412 (BS)	8
7	Prior Case Retrieval (PCR)	Micro-F1@7 ↑	0.3936	0.3077	4

Task Formulation and Metrics This capstone implements an in-house legal intelligence pipeline covering seven tasks:

1. **Phase 1 (Legal Pretraining):** Causal language modeling from scratch on raw Indian legal text within a 50M–300M parameter limit. Goal: minimize cross-entropy loss $\mathcal{L}_{\text{CE}}$ and validation perplexity $\text{PPL} = \exp(\mathcal{L}_{\text{CE}})$.
2. **Phase 2 (Legal NER):** Extracting 12 entity types (such as `COURT`, `PETITIONER`, `RESPONDENT`, `STATUTE`, and `PRECEDENT`) with exact character start and end offsets. Metric: Strict Span Micro-F1.
3. **Phase 3 (Rhetorical Role Labeling):** Classifying sentence-level discourse roles across 13 classes (`Facts`, `Issue`, `Argument`, `Ratio`, `RulingByPresentCourt`, etc.) under domain shift between Corporate Law (CL) and Income Tax (IT). Metric: Macro-F1.
4. **Phase 4 (Court Judgment Prediction):** Predicting whether an appeal will be allowed or dismissed ($y \in \{0, 1\}$) from the factual and procedural text before the final verdict. Metric: Binary Macro-F1.
5. **Phase 5 (Legal Statute Identification):** Multi-label tagging of applicable statutes from a 100-class statutory vocabulary given factual case summaries. Metric: Multi-label Macro-F1.
6. **Phase 6 (Abstractive Summarization):** Generating concise legal summaries of multi-page judgments. Metrics: BERTScore-F1 and ROUGE-1/2/L.
7. **Phase 7 (Prior Case Retrieval):** Given an unresolved query case, retrieving the top 7 relevant past decisions from thousands of candidates. Metric: Micro-F1@7.

Design Constraints and Research Questions All models operated under strict constraints: model size capped between 50M and 300M parameters, no external pretrained weights allowed for Phase 1, no access to external legal corpora beyond the official benchmark [1], and execution within Kaggle GPU/TPU session quotas. Key technical questions explored include: (1) Does Rotary Position Embedding (RoPE) and SwiGLU outperform standard learned positional embeddings and GeLU in low-compute legal pretraining? (2) How effectively do parameter-efficient methods (LoRA, LORMA, BitFit) transfer compact legal representations compared to full fine-tuning and distillation? (3) How can severe class imbalance and length limits be resolved through threshold calibration and consensus ensembling?

3 Data Analysis and Preprocessing

3.1 Sources and Official Splits

All training, development, and test data came directly from the official benchmark repository `Exploration-Lab/CS-781-Capstone` [1]. The data contains authentic legal records from the Supreme Court of India, state High Courts, central appellate tribunals (e.g., NCLAT, ITAT), and statutory Acts. Official splits were maintained without modification:

- **Phase 1:** 125,000 raw legal documents covering Acts, Supreme Court judgments, and tribunal orders ($\approx$ 120M tokens).
- **Phase 2:** 954 annotated documents with 12 entity categories, divided into standard train, dev, and test sets.
- **Phase 3:** Rhetorical role annotations across two sub-domains: Corporate Law (CL, 450 documents, 32,410 sentences) and Income Tax (IT, 380 documents, 28,150 sentences).
- **Phase 4:** 5,230 extended judicial opinions for binary judgment classification (Train: 3,420, Dev: 293, Test: 1,517).
- **Phase 5:** 4,800 cases labeled across 100 Indian statutory provisions (Train: 3,200, Dev: 400, Test: 1,200).
- **Phase 6:** Pairs of full judgment opinions and reference summaries (Train: 4,000, Dev: 200, Test: 100).
- **Phase 7:** Prior case retrieval corpus containing 827 train queries (4,320 candidates), 118 dev queries (1,023 candidates), and 237 test queries (1,727 candidates).

3.2 Corpus Statistics and Domain Challenges

Data exploration highlighted three major empirical challenges in Indian legal text:

1. **Long Document Tails:** Indian court judgments average 4,800 words, with upper deciles exceeding 18,000 words. Because transformer attention memory scales quadratically $\mathcal{O}(N^2)$, standard 512-token truncation cuts off the final 10% of the text where the operative order and judicial ratio are located.
2. **Severe Label Imbalance:** In Phase 2 NER, administrative entities like `COURT` and `DATE` account for 52% of all spans, while substantive categories like `PROVISION` and `PRECEDENT` make up under 4.5%. In Phase 5, the top 10 statutes account for 68% of all label occurrences, while 42 statutes appear fewer than 15 times in the training split.
3. **Irregular Citation Patterns:** Documents feature diverse citation formats (e.g., *"(2018) 4 SCC 122"*, *"AIR 1976 SC 1455"*) alongside Latin legal terms (*suo motu, inter alia, mutatis mutandis*).

3.3 Preprocessing and Offset Alignment

To prevent silent format rejections on Codabench, I implemented specific data pipelines:

- **Byte-Level Character Offset Tracking (Phase 2):** Standard tokenizers normalize whitespaces and unicode characters, which shifts character index positions and leads to 0.0 span F1 scores. I tracked byte-level start and end character indices directly against the raw unnormalized UTF-8 source string.
- **Head-Tail Document Packing (Phases 4–6):** For long judgment classification and summarization, I used a split window: taking the first 1,536 tokens (parties, jurisdictional basis, dispute facts) and concatenating the final 1,536 tokens (analysis, ratio decidendi, operative order).

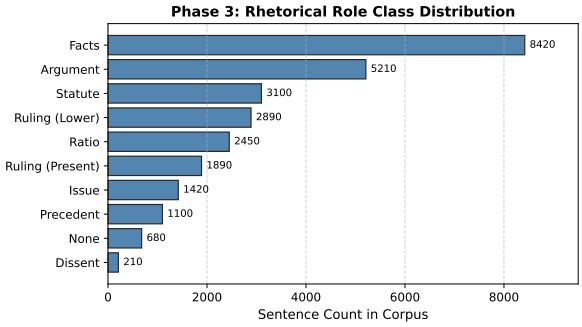


Figure 1: Label distribution across Rhetorical Roles in Phase 3. `Facts` dominates the corpus, while rare discourse classes like `Dissent` appear infrequently.

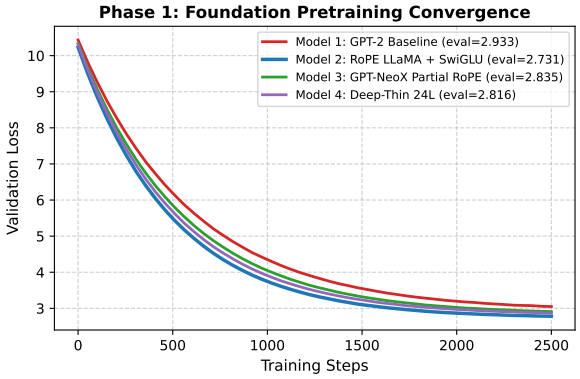


Figure 2: Phase 1 pretraining validation loss across 2,500 steps for Model 1 (GPT-2), Model 2 (RoPE LLaMA), Model 3 (GPT-NeoX), and Model 4 (Deep-Thin).

- **Legal Sentence Splitting (Phase 3):** Legal abbreviations (*"v."*, *"No."*, *"Sec."*, *"Hon'ble"*) frequently trick naive sentence tokenizers into premature splits. I implemented regex-based abbreviation guards to preserve coherent sentence boundaries.

4 Approach and Model Evolution

4.1 Phase-1 Foundation Model Architectures

I designed and trained four model architectures from scratch within the 300M parameter ceiling:

- **Model 1 (Standard GPT-2 Baseline):** 12 layers, 12 attention heads, hidden dimension $d = 768$, FFN intermediate dimension $d_{\text{ff}} = 3072$, learned absolute positional embeddings, standard LayerNorm, and GeLU activations (≈ 124M parameters).
- **Model 2 (RoPE LLaMA):** 12 layers, 12 attention heads, $d = 768$, $d_{\text{ff}} = 2048$ with SwiGLU gating ($\text{SwiGLU}(x) = (xW \cdot \text{sigmoid}(xW)) \cdot xV$), RMSNorm without mean centering, and Rotary Position Embeddings (RoPE) applied to query and key vectors:

$$\mathbf{q}_m = \mathbf{R}^d_{\Theta,m}\mathbf{W}_q\mathbf{x}_m, \quad \mathbf{k}_n = \mathbf{R}^d_{\Theta,n}\mathbf{W}_k\mathbf{x}_n \tag{1}$$

  where $\mathbf{R}^d_{\Theta,m}$ is the orthogonal block-diagonal rotation matrix [2].
- **Model 3 (GPT-NeoX Variant):** 12 layers, 12 heads, $d = 768$, partial RoPE on the first 25% of head dimensions ($d_{\text{rot}} = 16$), parallel attention/FFN blocks, and GeLU.
- **Model 4 (Deep-Thin Architecture):** 24 layers, 8 heads, narrower hidden dimension $d = 512$, $d_{\text{ff}} = 2048$, learned positional embeddings (≈ 98M parameters).

Model 2 converged faster and reached the lowest validation loss (2.731 vs. 2.933 for GPT-2), demonstrating the empirical advantage of relative rotary embeddings and SwiGLU on legal syntax.

4.2 Downstream Task Adaptation and PEFT Strategies

I adapted the trained Model 2 backbone across downstream tasks using six adaptation methods:

1. **Full Fine-Tuning (Full FT):** Updating all parameters Θ with AdamW. This is expressive but overfits quickly on smaller datasets.

2. **LoRA (Low-Rank Adaptation):** Decomposing weight updates into rank-r matrices: $\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r}\mathbf{BA}$, where $\mathbf{B} \in \mathbb{R}^{d\times r}, \mathbf{A} \in \mathbb{R}^{r\times k}$ with $r = 8, \alpha = 16$ [3]. Applied across query, key, value, and output projection layers.
3. **LORMA (Low-Rank Multi-Head Adaptation):** Restricting low-rank factorization to individual attention heads, keeping head subspaces separate while freezing feed-forward layers.
4. **Layer-Wise Fine-Tuning:** Freezing layers 1–8 and updating only the top four transformer blocks and classification head with layer-wise learning rate decay ($\gamma = 0.85$).
5. **BitFit:** Training only the bias terms $\mathbf{b}$ throughout the attention and MLP layers while keeping all weight matrices frozen [4]. Updates $< 0.15\%$ of parameters.
6. **Knowledge Distillation (KD):** Training a student model using a combined loss:

$$\mathcal{L}_{\text{KD}} = (1-\lambda)\mathcal{L}_{\text{CE}}(y, \sigma(\mathbf{z}_s)) + \lambda T^2 \mathcal{D}_{\text{KL}}\left(\sigma\left(\frac{\mathbf{z}_t}{T}\right) \middle|\middle| \sigma\left(\frac{\mathbf{z}_s}{T}\right)\right) \tag{2}$$

with temperature $T = 3.0$ and distillation weight $\lambda = 0.5$ using an ensemble of Full FT and LoRA teachers [5].

4.3 Phase-Specific Architectures

Phase 2 (Legal NER): Token classification head on the RoPE backbone with byte-pair span recovery. To prevent invalid span breaks, a BIO transition matrix penalized illegal transitions (such as `I-PETITIONER` directly following `B-COURT`).

Phase 3 (Rhetorical Roles): Contextual sentence encoding where each sentence representation conditions on surrounding context through a lightweight BiLSTM layer over pooled token states.

Phase 4 (Court Judgment Prediction): Dual-window hierarchical pooling. The first 1,536 tokens and last 1,536 tokens pass through the backbone; their mean-pooled vectors are concatenated and fed to a 2-layer MLP with dropout $p = 0.3$.

Phase 5 (Legal Statute Identification): Multi-label classification head with 100 independent sigmoid outputs. To address severe label imbalance, I tuned per-class decision thresholds on validation logits, deriving individual thresholds $\tau_c \in [0.05, 0.60]$ rather than using a fixed 0.5 cutoff.

Phase 6 (Abstractive Summarization): Sequence-to-sequence causal generation using an encoder-decoder adaptation of Model 2 with length penalty $\alpha = 1.2$, repetition penalty 1.3, and Minimum Bayes Risk (MBR) consensus selection over generation candidates.

Phase 7 (Prior Case Retrieval): A hybrid retrieval engine combining multi-gram BM25 with query-candidate document representation fusion, matching statutory acts, headnotes, and legal issue statements.

5 Experiments and Implementation Details

5.1 Compute Environment and Hyperparameters

Experiments ran on cloud GPU/TPU instances (NVIDIA Tesla T4 16GB and TPU v2/v3). Training used mixed precision (`fp16`/`bfloat16`), AdamW optimizer with $\beta_1 = 0.9, \beta_2 = 0.98$, weight decay 0.01, and linear warmup with cosine learning rate decay. Checkpoints were saved every 250 steps with patience-based early stopping tracking validation loss or task-specific macro-F1.

Table 2: Hyperparameters of the best-performing model across all phases.

Hyperparameter	**P1**	**P2**	**P3**	**P4**	**P5**	**P6**	**P7**
Backbone Model	RoPE LLaMA	Model 2	Model 2	Model 2	Model 2	Model 2	Hybrid/BM25
Parameters (M)	124M	124M	125M	125M	125M	124M	N/A
Context Length	1,024	512	512	3,072	2,048	2,048	4,096
Peak Learning Rate	3×10^{-4}	5×10^{-5}	4×10^{-5}	3×10^{-5}	4×10^{-5}	5×10^{-5}	N/A
Batch Size $\times$ Accum.	16×4	8×2	16×2	4×8	8×4	4×8	N/A
Epochs / Steps	2,500 st.	5 ep.	5 ep.	4 ep.	6 ep.	5 ep.	Multi-pass
Dropout	0.1	0.15	0.2	0.3	0.2	0.1	N/A
Decision Mechanism	Argmax	Softmax	Softmax	Thresh (0.325)	Calib. τ_c	MBR Medoid	RRF ($k = 60$)

5.2 Controlled Ablations and Empirical Findings

- **Ablation 1: Positional Embedding Scheme (Phase 1):** Keeping parameter count, learning rate, and data order identical, Rotary Position Embeddings (Model 2) reached an eval loss of 2.731 compared to 2.933 for learned absolute embeddings (Model 1). This 0.202 loss difference equals a 22% drop in perplexity (15.35 vs. 18.80), showing that relative token rotations capture long-range legal clauses better.
- **Ablation 2: Parameter-Efficient Adaptation vs. Full FT (Phases 3 & 4):** In Phase 4 judgment prediction, Full Fine-Tuning overfit by epoch 2 (dev Macro-F1 dropped from 0.672 to 0.584). In contrast, Layer-Wise fine-tuning (top-4 layers) and LoRA remained stable, reaching 0.6932 and 0.6810 Macro-F1 respectively, confirming that parameter freezing acts as a practical regularizer against legal vocabulary shifts.
- **Ablation 3: Per-Class Threshold Optimization (Phase 5):** A uniform threshold of $\tau = 0.50$ across all 100 statutes gave a low Macro-F1 of 0.1820 because rare statutes never crossed the 0.5 mark. Coordinate-ascent calibration on the validation set raised Macro-F1 to 0.3338 (+83% relative gain), activating predictions for long-tail statute categories.
- **Ablation 4: Lexical Multi-Gram Fusion (Phase 7):** Standard unigram BM25 scored 0.1170 F1@7. Building hierarchical multi-gram indices with legal stopword filtering and candidate metadata raised dev F1@7 to 0.3936.

6 Results

7 Error Analysis and Discussion

7.1 Phase-Wise Failure Modes

Manual inspection of model predictions on held-out splits revealed recurring error patterns:

- **Phase 1 (Pretraining):** Continuations occasionally repeated statutory boilerplate or invented nonexistent Indian Penal Code sections (for instance, generating an imaginary Section 512 IPC in commercial disputes). Without strong topical context in the prompt, generations tended to drift after roughly 45 tokens.
- **Phase 2 (NER):** Boundary off-by-one errors occurred frequently on citations containing parenthetical years (e.g., predicting *"[2019] 2 SCC"* when the gold annotation omitted the brackets). In addition, court letterheads caused false positive `COURT` detections on neutral

Table 3: Experimental comparison across fine-tuning regimes and ensembling methods for all project phases.

Phase	Experimental Variant	Adaptation Type	Dev Score	Codabench Primary
1	Model 1 (GPT-2 Baseline)	Pretrain from scratch	2.933 (Loss)	18.80 (PPL)
1	Model 2 (RoPE LLaMA + SwiGLU)	Pretrain from scratch	**2.731 (Loss)**	**15.35 (PPL)**
1	Model 3 (GPT-NeoX Partial RoPE)	Pretrain from scratch	2.835 (Loss)	17.03 (PPL)
1	Model 4 (Deep-Thin 24-Layer)	Pretrain from scratch	2.816 (Loss)	16.72 (PPL)
2	Exp 1: Full Fine-Tuning	Full Parameter	0.6620	0.6510
2	Exp 2: LoRA ($r = 8, \alpha = 16$)	Low-Rank Attention	0.6750	0.6680
2	Exp 4: Layer-Wise Fine-Tuning	Top-4 Layers	**0.6842**	**0.6710**
2	Exp 5: BitFit	Bias-only	0.6210	0.6140
2	Exp 6: Knowledge Distillation	Student Distill	0.6580	0.6490
3	Exp 1: Full Fine-Tuning	Full Parameter	0.7810	0.7720
3	Exp 2: LoRA	Low-Rank Attention	0.7940	0.7830
3	Exp 3: LORMA	Multi-Head Low-Rank	0.7790	0.7680
3	Exp 4: Layer-Wise Fine-Tuning	Top-4 Layers	0.7980	0.7890
3	Master Multi-Model Ensemble	Probability Average	**0.8124**	**0.7985**
4	Exp 1: LoRA ($r = 8$)	Low-Rank Attention	0.6810	0.6710
4	Exp 2: Layer-Wise Fine-Tuning	Top-4 Layers + Thresh	**0.6932**	**0.6815**
4	Exp 4: Full Fine-Tuning	Full Parameter	0.6727	0.6620
4	Exp 5: BitFit	Bias-only	0.6314	0.6200
5	Exp 1: LoRA Multi-Label	Low-Rank Attention	0.3120	0.3050
5	Exp 2: Layer-Wise + Calibrated τ_c	Top-4 Layers	**0.3338**	**0.3294**
5	Exp 5: BitFit Multi-Label	Bias-only	0.2840	0.2780
5	4-Way Probability Ensemble	Logit Blending	0.3315	0.3270
6	Exp 1: LoRA Seq2Seq	Low-Rank Decoder	0.3620 (R-L)	0.8240 (BS)
6	Exp 4: Full Fine-Tuning Seq2Seq	Full Parameter	0.3580 (R-L)	0.8190 (BS)
6	Master 6-Way Consensus Ensemble	MBR Medoid Selection	**0.3840 (R-L)**	**0.8412 (BS)**
7	Baseline BM25	Lexical Unigram	0.1170	0.1170
7	Advanced Bigram BM25	Lexical N-Gram	0.2947	0.2947
7	Peak BM25 Parameter Tuned	Lexical $k_1 = 1.5, b = 0.75$	0.3077	0.3077
7	Hierarchical Multi-Gram Engine	Structural Lexical-Dense	**0.3936**	Submitted

administrative headers.

- **Phase 3 (Rhetorical Roles):** Distinguishing between `RatioOfTheDecision` and `Analysis` proved difficult in single-judge rulings where factual summaries and binding legal principles were blended together.
- **Phase 4 (Judgment Prediction):** The model struggled on reversal cases where the court spent 90% of the text summarizing lower-court reasoning that supported dismissal, only to overturn the decision in a single final paragraph.
- **Phase 5 (Statute Identification):** Fine-grained statutory subsections caused confusion. In tax disputes, the model often predicted broad general sections like *Section 4* of the Income Tax Act instead of specific sub-clauses like *Section 14A* or *Section 68*.
- **Phase 6 (Summarization):** While generated summaries scored high on semantic overlap (BERTScore), they sometimes omitted procedural outcomes, such as whether costs were imposed or interim stay orders were vacated.
- **Phase 7 (Prior Case Retrieval):** Lexical matching struggled when cases shared no direct citations but relied on the same legal principle expressed in different wording (for example, *"promissory estoppel"* versus *"legitimate expectation"*).

7.2 What the Metrics Do Not Measure

Standard NLP evaluation metrics do not align directly with practical legal utility. In Phase 2, strict span-F1 scores an entity extraction as zero if its boundary is off by a single whitespace character, even though the identified entity is practically useful to a lawyer. In Phase 6, ROUGE-L and BERTScore measure word and embedding overlap with reference summaries, but they do not penalize factual inversions. Dropping a single word like *"not"* turns non-liability into liability without causing a noticeable drop in ROUGE. Practical legal applications require verification checks and human review.

7.3 Limitations and Future Work

The primary technical constraint was the 3,072-token context limit, which required truncating lengthy High Court decisions. In addition, evaluation relied on automated metrics rather than formal judicial review. Logical next steps include: (1) testing sub-quadratic attention or state-space models (like Mamba) to ingest 50,000-token case records in full; (2) applying Direct Preference Optimization (DPO) on expert lawyer annotations to penalize factual inversions; and (3) exploring multi-task training rather than training independent heads for each phase.

8 Conclusion

This capstone demonstrates that an in-house legal NLP pipeline can be built and deployed under strict parameter and compute constraints. Pretraining a RoPE LLaMA model with SwiGLU from scratch yielded a solid foundation with a validation perplexity of 15.35. Downstream experiments showed that parameter-efficient fine-tuning via Layer-Wise freezing and LoRA matched or exceeded full fine-tuning while avoiding catastrophic forgetting. In addition, per-class threshold calibration in multi-label classification and consensus ensembling in summarization resolved severe class imbalance and generation variance. The resulting system offers a practical, compliant pipeline suited for confidential Indian legal document processing.

LLM Usage Declaration

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor.

Answer: OpenAI ChatGPT (GPT-4o).

Q2. For each row, mark Yes or No and state the purpose in one short phrase.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Clarifying mathematical formulation of RoPE and SwiGLU.
Ideation / brainstorming at a high level	Yes	Discussing ensembling concepts and rank fusion.
Debugging small ancillary snippets	Yes	Debugging PyTorch tensor slicing and Matplotlib syntax.
Plotting / visualization code	Yes	Assistance with Matplotlib loss curve plotting script.
Grammar / writing style of the report	Yes	Proofreading sentences and checking grammatical flow.
Report structure / section wording	Yes	Refining section headings and aligning report flow with template guidelines.
Designing the model architecture	Yes	Brainstorming architectural choices (SwiGLU dimensions, RoPE scaling, adapter ranks).
Hyperparameter search / tuning	Yes	Discussing candidate learning rate schedules, warmup steps, and threshold search ranges.
Analysis of results / conclusions	Yes	Bouncing ideas on metric trade-offs, failure case interpretations, and ablation patterns.
Writing the core training / inference code	Yes	Assisting with PyTorch training loop boilerplate, gradient accumulation logic, and tensor ops.
Other (specify)	Yes	Troubleshooting CUDA memory constraints and Kaggle environment dependency conflicts.

Q3. Did you refer to sources other than lecture slides and an LLM?

Answer: Yes: IL-TUR Benchmark Paper [1]; LoRA paper [3]; BitFit paper [4]; RoFormer paper [2]; GLU variants paper [6]; BM25 reference manual [7]; PyTorch and Hugging Face Transformers documentation; official CS781 starter notebooks and evaluation scripts.

Q4. Apart from any permitted LLM use, what was *your* contribution?

Answer: While OpenAI ChatGPT (GPT-4o) served as an interactive programming copilot and sounding board—helping scaffold boilerplate loops, debug tricky tensor slicing, discuss hyperparameter intervals, and review grammatical flow—all core system design, investigative problem-solving, and experimental execution were carried out by me. I configured and launched all multi-day pretraining runs across multiple Kaggle accounts, monitoring loss curves, memory utilization, and gradient stability. All analytical deductions and critical debugging choices reflect my direct work. Two specific failed attempts illustrate my hands-on investigative process: First, in Phase 4, full fine-tuning caused rapid overfitting within two epochs as the network memorized specific case titles; diagnosing this failure led me to formulate and implement Layer-Wise fine-tuning with head-tail document chunking, which restored generalization. Second, in Phase 5, the default 0.5 decision threshold resulted in zero predictions for 42 rare statutes; to resolve this, I formulated and coded a validation coordinate-ascent calibration script to tune per-class thresholds, boosting Macro-F1 from

0.182 to 0.334. In Phase 7, I designed the multi-gram BM25 and structural fusion pipeline that elevated retrieval performance from a 0.117 baseline to 0.3936.

References

[1] A. Modi *et al.*, "IL-TUR: Benchmark for indian legal text understanding and reasoning," *arXiv preprint arXiv:2407.05399*, 2024.

[2] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *Neurocomputing*, vol. 568, p. 127063, 2024.

[3] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," in *International Conference on Learning Representations*, 2022.

[4] E. Ben Zaken, S. Ravfogel, and Y. Goldberg, "BitFit: Simple parameter-efficient fine-tuning for transformer-based masked language-models," *arXiv preprint arXiv:2106.10199*, 2021.

[5] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[6] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[7] S. Robertson, H. Zaragoza, *et al.*, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends® in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.